Memetic Algorithm with NEH Seeding and Tabu-based Local Search for Permutation Flowshop Scheduling

結合 NEH 初始化與 Tabu 區域搜尋之迷因演算法於定序流水線排程問題之研究

1st Hung Ming-Chun
Department of Computer Science and Information Engineering
National Taiwan Normal University
Taipei, Taiwan(R.O.C)
61447026s@ntnu.edu.tw

2nd Hsieh Yao-yu
Graduate Institute of Science Education
National Taiwan Normal University
Taipei, Taiwan(R.O.C)
61345001s@ntnu.edu.tw

3rd Given Name Surname
Department of Computer Science and Information Engineering
National Taiwan Normal University
Taipei, Taiwan(R.O.C)
@ntnu.edu.tw

***Abstract*—**

I. Introduction

定序流水線排程問題（Permutation Flowshop Scheduling Problem, PFSP）是排程領域中經典且具高度實務價值的組合最佳化問題，廣泛應用於製造系統、生產排程與資源配置等實務問題中。此問題的目標是在多台機器上決定所有工作的加工順序，且所有機器需依相同順序處理工作，使總完工時間（makespan）最小化。由於 PFSP 在一般情況下屬於 NP-hard 問題，當工作數與機器數增加時，傳統精確演算法往往難以在合理時間內求得最佳解。

基因演算法（Genetic Algorithm, GA）因具備良好的全域搜尋能力與彈性的排列編碼方式，常被用於求解 PFSP。然而，傳統 GA 在演化後期容易因族群多樣性下降而產生早熟收斂，使得搜尋停滯於區域最佳解附近。為了同時兼顧全域探索能力與局部開發能力，本研究採用迷因演算法（Memetic Algorithm, MA）的觀點，以 GA 作為族群式搜尋主體，並結合軌跡式元啟發法作為強化局部搜尋能力的改良機制。

本研究使用 permutation-based encoding 表示解，並設計一個特殊初始化程序與軌跡式元啟發法的整合架構。具體而言，我們首先以 NEH 建構式啟發法產生高品質初始種子，以改善初始族群品質；接著比較多種適用於排列問題的交配算子與突變設定；最後導入以 swap 為鄰域的 Tabu-based local search，針對菁英個體進行局部強化，形成一個用於 PFSP 的 memetic framework。

本研究的目的不僅在於建構一個有效的 PFSP 求解方法，同時亦系統性分析不同設計選擇對演算法效能之影響，包括交配算子、突變率、突變方式、NEH 初始化，以及 Tabu-based local search 的整合方式。實驗將使用 Taillard benchmark 測資，並透過多次獨立執行比較各方法在解品質、穩定性與運算成本上的差異。

II. Problem definition

本研究考慮定序流水線排程問題（Permutation Flowshop Scheduling Problem, PFSP）。設共有 n 個工作與 m 台機器，所有工作皆須依固定順序經過各台機器加工，即每個工作都必須依序通過 $M_1, M_2, \ldots, M_m$。此外，每台機器在任一時間僅能處理一個工作，且加工過程不可中斷。

PFSP 的特徵在於所有機器皆以相同的工作順序進行加工。換言之，若某一解表示為一個工作排列 $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$，則此排列同時決定所有機器上的加工順序。因此，本問題的核心在於尋找一組最佳排列，使所有工作完成加工的最終時間最小。

令 p_{i,π_k} 表示工作 π_k 在機器 M_i 上的加工時間，$C(\pi_k, i)$ 表示工作 π_k 在機器 M_i 上的完工時間，則其遞迴關係可表示為

$$C(\pi_k, i) = \max\{C(\pi_{k-1}, i),\ C(\pi_k, i-1)\} + p_{i,\pi_k}.$$

其中，$C(\pi_0, i) = 0$ 與 $C(\pi_k, 0) = 0$。本研究之目標函數為最小化總完工時間（makespan）：

$$C_{\max} = C(\pi_n, m).$$

因此，PFSP 可視為一個在所有可行工作排列中，尋找使 $C_{\max}$ 最小之最佳排列的組合最佳化問題。

III. Proposed Memetic Algorithm

A. Solution Representation

本研究採用排列式編碼表示解。若共有 n 個工作，則一個候選解可表示為一個長度為 n 的排列向量，其中每個元素對應一個唯一的工作編號，且每個工作僅出現一次。

例如，當一個解表示為 $(3, 6, 4, 1, 5, 2)$ 時，代表所有機器皆依照工作 $3 \to 6 \to 4 \to 1 \to 5 \to 2$ 的順序進行加工。由於 PFSP 要求所有機器採用相同的工作順序，因此此種排列式編碼方式可直接對應到一個合法排程，並適合搭配排列問題常見的交配、突變與區域搜尋運算子。

B. Fitness Evaluation

本研究以總完工時間作為解的適應值評估指標。對於任一給定的工作排列，系統會依序模擬各工作在每台機器上的加工流程，並計算最後一個工作於最後一台機器上的完工時間，作為該解的 makespan。

在實作上，我們設計了一個排程評估函數。令 `machine_end_time[i]` 表示機器 M_i 的最早可用時間，當某一工作進入機器 M_i 時，其開始時間取決於該工作在前一台機器上的完工時間與該機器目前最早可用時間兩者中的較大值。完成所有工作與所有機器的模擬後，最後一台機器的完工時間即為該排列的 makespan。

C. Genetic Framework

本研究以基因演算法作為迷因演算法的主體架構。演算法維護一組族群，並透過選擇、交配與突變逐代演化，以持續改善解的品質。

在親代選擇方面，本研究採用錦標賽選擇法。每次由族群中隨機挑選若干個體，並選出其中 makespan 最小者作為親代，以兼顧選擇壓力與族群多樣性。在世代更新方面，本研究採用菁英保留策略，將當代最佳個體直接保留至下一代，以避免優良解在交配或突變過程中流失。

此外，本研究以目標函數評估次數（function evaluations, FFE）作為演算法的主要搜尋預算控制方式。每當候選解被評估一次時，FFE 計數器即增加 1，並於達到預設上限時終止演化流程。

D. Crossover and Mutation Operators

由於本研究採用排列式編碼表示解，傳統的單點或多點交配無法直接套用，否則容易產生重複工作或遺漏工作的非法排列。因此，本研究實作了四種常見的排列交配算子，分別為順序交配（Order Crossover, OX）、線性順序交配（Linear Order Crossover, LOX）、部分映射交配（Partially Mapped Crossover, PMX）以及循環交配（Cycle Crossover, CX），並比較其在 PFSP 上的效能表現。

OX 會先保留父代的一段連續基因片段，再依照另一個父代中的相對出現順序填補其餘空位。此方法能在保留部分局部順序資訊的同時，維持子代為合法排列。

LOX 與 OX 相似，同樣先保留一段父代片段，但其餘位置的填補方式採用由左至右的線性順序，而非循環繞回。此特性使其更強調線性結構的保存。

PMX 則透過兩個切點建立父代片段間的對應關係，並利用映射規則修正其餘位置的衝突，以同時兼顧絕對位置與相對結構的保留。

CX 的核心概念是在父代之間找出循環結構，使子代各位置上的基因來自其中一個父代在相同位置上的基因，因此特別強調絕對位置資訊的保留。

在突變方面，本研究考慮兩種排列問題常見的突變方式：交換突變 (Swap) 與插入突變 (Insertion)。交換突變會隨機選擇兩個位置並交換其工作；插入突變則會抽出一個工作並插入到另一個位置。透過比較不同突變方式與突變率設定，可分析其對族群多樣性與局部搜尋能力的影響。

E. NEH-based Initialization

本研究在初始族群中引入 NEH 建構式啟發法所產生的高品質種子解。NEH 方法首先計算各工作在所有機器上的總加工時間，並依總加工時間由大至小排序；接著依序將工作插入目前部分序列的最佳位置，以逐步建構一組品質較佳的初始解。

在本研究中，初始族群並非完全隨機生成，而是於族群中注入一個由 NEH 產生的種子個體，其餘個體則維持隨機排列。此設計可在保留族群多樣性的同時，提升演化初期的搜尋品質與收斂效率。

藉由這種逐一探索局部最佳解的插入方式，NEH 能夠產生結構極佳的初始解。將此 NEH 解注入基因演算法的初始族群中，能確保演算法從一個具備優良基因片段的起點開始演化，進而顯著提升最終搜尋到全局最佳解的機率與效率。

F. Tabu-based Local Search

為了提升演算法的局部開發能力，本研究於基因演化流程中加入 Tabu-based 區域搜尋，形成迷因演算法架構。區域搜尋以交換鄰域為基礎，亦即對目前解中任兩個位置的工作進行交換，以產生鄰近解。

在每一世代完成族群更新後，本研究選取當代最佳個體作為區域搜尋的對象，並對其執行固定步數的禁忌搜尋。搜尋過程中，系統以交換的工作對作為禁忌移動記錄，避免搜尋在短時間內重複走回已探索區域；若某禁忌移動可產生優於目前歷史最佳解的結果，則透過特赦準則予以接受。區域搜尋完成後，改良後的個體會回寫至族群中，並作為後續世代演化的基礎。

此設計使演算法能以 GA 維持全域探索能力，並以禁忌搜尋加強對高品質個體的局部精修，藉此兼顧 exploration 與 exploitation。

綜合而言，本研究所提出的迷因演算法以基因演算法作為全域搜尋主體，並結合 NEH 初始化與 Tabu-based 區域搜尋，以同時提升演化初期的解品質與後期的局部開發能力。此架構兼顧探索與開發，作為後續實驗比較的主要方法。

G. Population-level Tabu Mechanism

除了將 Tabu Search 用於菁英個體的局部改良外，本研究亦曾於族群演化過程中加入一個 population-level tabu mechanism，用以避免短時間內重複產生相同或過於相似的排列。具體而言，本機制會將已產生之個體以雜湊值形式記錄，並於 offspring 生成時檢查其是否已存在於當前族群或近期 tabu list 中。若某候選個體被判定為 tabu，則該個體不會直接加入下一代，而是重新嘗試產生新的 offspring；若多次嘗試後仍無法得到合法且非 tabu 的候選解，則以隨機個體補入族群。

此機制的目的在於降低族群中重複解的比例，並提升搜尋過程中的多樣性。然而，與針對個體進行局部精修的 Tabu-based local search 不同，此處之 tabu 機制主要作用於族群層級，屬於一種 diversity control strategy，而非局部搜尋程序本身。

IV. Experiment Design

A. Experimental Settings

本研究使用 Taillard benchmark 作為測試資料集，並對每一組演算法設定進行 20 次獨立執行，以降低隨機因素對結果的影響。除非另有說明，各組實驗皆採用相同的基本設定，以確保比較的公平性。

在演化參數方面，本研究設定族群大小為 50，交配率為 0.8，突變率則依實驗目的設定為不同數值。親代選擇採用錦標賽選擇法 (k-tournament)，其錦標賽大小設定為 3，並於每一世代保留當代最佳個體。對於區域搜尋部分，Tabu-based local search 之最大搜尋步數與禁忌期限則依預設設定執行。

此外，本研究以目標函數評估次數 (FFE) 作為主要停止條件，以確保不同方法之比較具有公平性。設族群大小為 50，則本研究將最大 FFE 設定為 50000。此外，由於加入區域搜尋後，每一世代所需的評估數可能顯著增加，因此本研究亦額外記錄實際完成的世代數，作為分析演算法搜尋成本與演化效率的輔助指標。

B. Comparison Plan

為了系統性分析各設計選擇對演算法效能的影響，本研究依序進行以下比較。首先，比較不同交配算子在相同突變與初始化設定下的表現，以選出較適合作為後續實驗基礎的交配方式。其次，在固定交配算子的情況下，比較不同突變率與突變方式，以分析其對搜尋品質與穩定性的影響。

在決定較佳的基因演算法設定後，本研究進一步比較是否使用 NEH 初始化，以分析特殊初始化程序對演化初期品質與最終結果的影響。最後，在前述較佳設定下加入 Tabu-based local search，並與未使用區域搜尋的版本進行比較，以驗證迷因演算法架構是否能進一步提升解品質。

C. Performance Measures

本研究在比較不同方法時，記錄各組實驗於 20 次獨立執行下的最佳值、平均值、最差值與標準差。此外，亦統計平均評估次數與平均執行時間，以評估各方法之運算成本。若可取得最佳已知解（best known solution, BKS），亦進一步計算平均偏差百分比，以衡量解品質與最佳已知解之差距。若已知某測資之 BKS，則本研究進一步計算平均偏差百分比（average deviation percentage）：

$$\text{Avg. Dev. (\%)} = \frac{\text{Avg} - \text{BKS}}{\text{BKS}} \times 100\%.$$

V. Results and Discussion

A. Effect of Crossover Operator

為了分析不同交配算子對演算法效能之影響，本實驗固定族群大小、交配率、突變率與突變方式，並僅改變交配算子種類。具體而言，本實驗固定使用插入突變，突變率設為 0.1，且暫不啟用 NEH 初始化與 Tabu-based local search，以公平比較 OX、LOX、PMX 與 CX 四種交配算子之表現。

表 I 彙整了不同交配算子在九組 Taillard 測資上的平均 makespan。若以平均解品質作為主要比較指標，LOX 在多數測資中取得較佳結果，整體表現最為穩定。PMX 與 OX 在部分測資中亦能取得接近 LOX 的結果，而 CX 雖然在若干案例中也具有競爭力，但整體平均表現仍略遜於 LOX。

綜合而言，LOX 在解品質與穩定性之間取得較佳平衡，因此本研究後續實驗將以 LOX 作為主要交配算子，並在此基礎上進一步比較突變率、突變方式、NEH 初始化以及 Tabu-based local search 的效果。包含最小值、最大值與標準差的完整結果整理於附錄表 VI。

TABLE I: 在不同交配算子下的平均 makespan

Instance	OX	LOX	PMX	CX
tai20_5_1	1295.85	1296.05	**1294.60**	1297.10
tai20_10_1	1627.55	**1621.85**	1623.40	1629.55
tai20_20_1	2373.00	2376.05	**2360.80**	2363.25
tai50_5_1	2742.85	**2741.35**	2742.80	2744.75
tai50_10_1	**3145.70**	3152.65	3150.80	3150.05
tai50_20_1	4056.80	**4052.85**	4054.40	4057.50
tai100_5_1	**5512.90**	5517.20	5516.15	5513.15
tai100_10_1	5963.00	**5939.00**	5970.60	5959.85
tai100_20_1	6687.35	6685.45	**6674.90**	6679.70

B. Effect of Mutation Settings

在決定以 LOX 作為主要交配算子後，本研究進一步分析突變設定對演算法效能之影響。考量突變率與突變方式之間可能存在交互作用，本研究同時比較插入突變與交換突變兩種方式，並搭配兩種突變率（0.1 與 0.5）進行實驗。其餘參數皆維持一致，且暫不啟用 NEH 初始化與 Tabu-based local search。

表 II 彙整了四種突變設定在九組 Taillard 測資上的平均 makespan。由結果可觀察到，突變率對整體表現具有顯著影響。在多數測試案例中，較高的突變率（0.5）在插入突變下能取得最佳平均解品質，顯示較強的擾動有助於維持族群多樣性並避免搜尋過早收斂。

綜合而言，插入突變搭配突變率 0.5 在本研究中展現最佳整體表現，因此後續實驗將採用此設定作為預設突變策略。

TABLE II: 在不同突變組合設定下的平均 makespan

Instance	Insertion + 0.1	Insertion + 0.5	Swap + 0.1	Swap + 0.5
tai20_5_1	1293.35	**1291.30**	1297.80	1297.00
tai20_10_1	1624.35	**1603.65**	1640.80	1617.65
tai20_20_1	2358.40	**2341.05**	2381.40	2346.30
tai50_5_1	2743.70	2734.80	2743.75	**2729.20**
tai50_10_1	3145.95	**3116.25**	3157.45	3139.20
tai50_20_1	4073.40	**4011.25**	4089.00	4051.35
tai100_5_1	5511.75	**5496.10**	5512.20	5506.80
tai100_10_1	5962.25	**5859.90**	5993.90	5906.55
tai100_20_1	6685.95	**6579.90**	6690.70	6581.90

C. Effect of NEH Initialization

在確定交配算子與突變設定後，本研究進一步分析 NEH 初始化對演算法效能之影響。為了公平比較，本實驗固定使用 LOX 作為交配算子，並採用插入突變搭配突變率 0.5；此外，Tabu-based local search 暫不啟用。在此設定下，比較是否使用 NEH 初始化對解品質的影響。

表 III 彙整了在有無 NEH 初始化下，各測試案例的平均 makespan。由結果可觀察到，在大多數測試案例中，使用 NEH 初始化皆能有效提升解品質，且在中大型問題（如 50-job 與 100-job）上改善特別明顯。例如在 tai100_10_1 與 tai100_20_1 等測資上，NEH 初始化可顯著降低 makespan，顯示其能提供較高品質的初始解。

然而，在少數案例中（如 tai50_10_1），NEH 初始化未能帶來明顯優勢，甚至略微劣於隨機初始化。此現象可能與演算法在演化過程中的探索能力有關，當族群已具備足夠多樣性時，初始解品質對最終結果的影響可能較為有限。

整體而言，NEH 初始化能在多數情況下提供較佳的起始解，並加速演算法收斂至高品質解，因此本研究後續實驗將採用 NEH 初始化作為預設設定。

TABLE III: 有無 NEH 初始化之平均 Makespan

Instance	Without NEH	With NEH
tai20_5_1	1291.30	**1279.20**
tai20_10_1	1607.20	**1598.00**
tai20_20_1	2338.00	**2327.05**
tai50_5_1	2735.85	**2728.75**
tai50_10_1	**3110.85**	3117.20
tai50_20_1	4019.40	**3972.10**
tai100_5_1	5499.85	**5495.75**
tai100_10_1	5879.85	**5825.15**
tai100_20_1	6574.80	**6505.00**

D. Effect of Tabu-based Local Search

E. With or without Tabu Mechanism in GA

在基因演算法中，族群的早熟收斂（Premature Convergence）一直是個棘手的問題。為了解決此問題，我們最初的假設是：若將禁忌機制（Tabu Mechanism）與同代重複基因排除機制引入 GA，強迫族群維持高度的遺傳多樣性，理應能獲得更佳的全局搜尋結果。為了驗證此假設，我們以順序交配（OX）為基礎，對比了「原始 GA」與「結合 Tabu 機制之 GA」的效能。實驗數據如表 IV 所示。

TABLE IV: GA (OX) 與 GA (OX) 加上 Tabu 機制之結果比較

Instance	**GA (OX)**			**GA (OX) with Tabu**		
	Min	Avg	Max	Min	Avg	Max
tai20_5_1	1283	1300.80	1324	**1282**	**1296.65**	**1305**
tai20_10_1	**1599**	1651.70	1697	**1599**	**1634.95**	**1685**
tai20_20_1	2342	2400.90	2459	**2334**	**2376.40**	**2437**
tai50_5_1	**2729**	**2743.65**	**2752**	**2729**	2746.00	2762
tai50_10_1	**3128**	**3172.60**	**3220**	3144	3227.25	3312
tai50_20_1	**4043**	**4089.40**	**4179**	4098	4176.45	4273
tai100_5_1	**5493**	**5512.75**	**5558**	5524	5540.20	5574
tai100_10_1	**5904**	**5984.30**	**6114**	6044	6113.05	6177
tai100_20_1	**6597**	**6729.95**	**6856**	6780	6890.65	6993

出乎意料的實驗結果：
從表 IV 可以觀察到一個與預期完全相反的現象。除了在 `tai20_20_1` 等少數小型測資中，Tabu 機制略微提升了搜尋品質外，在所有中大型測資（`tai50` 與 `tai100` 系列）中，**加入 Tabu 機制的 GA 表現全面劣於原始 GA**，其平均完工時間（Avg）甚至出現了顯著的退化。

反常現象之學術探討 (Over-diversification vs. Exploitation)：
深入分析演算法的行為軌跡，我們認為此現象起因於「過度多樣化（Over-diversification）」摧毀了 GA 的「局部開發（Exploitation/Intensification）」能力。

在原始的 GA 中，當搜尋進入中後期，族群會自然地收斂到幾個優良的解附近（盆地）。這些相似的高適應度個體（可能包含重複的雙胞胎）會主導交配池，透過微小的突變與重組，進行極其細緻的局部尋優，這是演算法能找到最終最佳解的關鍵步驟。

然而，我們設計的 Tabu 機制嚴格禁止同一代出現重複的 Hash 值。當族群開始收斂時，演算法會因為頻繁產生相似的子代而觸發碰撞。根據實作邏輯，當重試 10 次仍無法產生獨特個體時，系統會強制觸發「隨機移民（Random Immigration）」，引入完全隨機的全新排列以填補族群空缺。

對於 $N = 20$ 的小型問題，隨機生成的排列與優良解的距離不至於過度遙遠，偶爾的隨機擾動反而能幫助跳脫區域最佳解；但對於 $N = 50$ 或 $N = 100$ 的大型問題，其解空間呈現階乘級別的爆炸增長。此時，強制塞入的隨機個體其 Makespan 通常極度糟糕，形同於在族群中不斷注入無用的「雜訊」。為了滿足嚴格的「不重複」約束條件，Tabu-GA 不斷丟棄那些本應被用來進行局部細緻開發的優良子結構，把運算資源浪費在評估劣質的隨機解上。最終導致演算法失去了收斂與微調的能力，搜尋效率大幅衰退。也因此我們導入了動態的 Dynamic Tabu Tenure 機制，試圖在搜尋的不同階段調整禁忌期限，以期在保持多樣性的同時不至於完全犧牲局部開發能力。

F. Dynamic Tabu Tenure Mechanism in GA

為了解決靜態 Tabu 造成的「過度多樣化」以及喪失局部開發能力的問題，我們進一步實作了動態禁忌期限（Dynamic Tabu Tenure）機制。

退火式衰減策略 (Annealing-like Decay Strategy)：
仔細審視演化過程的兩難，我們發現演化初期需要極高的多樣性來探索解空間（Exploration），而演化後期則需要高度的集中力來微調收斂（Exploitation）。因此，實作中採用了類似模擬退火的線性衰減策略：在演化初期，賦予較長的禁忌期限（初始 `tabu_tenure = 10`），嚴格禁止重複基因以強迫拓荒；隨著世代數（Generation）推進，禁忌期限逐步遞減；直到演化進入最終階段時，禁忌長度降為 0 並徹底關閉 Tabu 機制，使演算法能毫無阻礙地對尋獲的優良盆地進行純粹的局部微調。

實驗結果如表 **??** 所示，我們對比了固定禁忌長度（Static Tabu）與動態禁忌長度（Dynamic Tabu）在順序交配（OX）下的表現。

TABLE V: GA (OX) Static Tabu 與 Dynamic Tabu 機制之結果比較

Instance	**GA (OX) with Static Tabu**			**GA (OX) with Dynamic Tabu**		
	Min	Avg	Max	Min	Avg	Max
tai20_5_1	**1297**	1299.00	1324	**1297**	**1298.05**	**1318**
tai20_10_1	**1597**	1639.40	**1669**	1599	**1637.50**	1675
tai20_20_1	2346	2385.35	2456	**2313**	**2365.85**	**2423**
tai50_5_1	**2729**	2743.90	2763	**2729**	**2743.75**	**2752**
tai50_10_1	3178	3234.00	3268	**3152**	**3181.80**	**3266**
tai50_20_1	4089	4157.25	4203	**4003**	**4085.80**	**4141**
tai100_5_1	5495	5533.90	5567	**5493**	**5519.25**	**5546**
tai100_10_1	5997	6107.60	6174	**5902**	**5990.75**	**6042**
tai100_20_1	6797	6892.10	6991	**6695**	**6754.20**	**6857**

效能全面反轉與收斂修復：
觀察表 V 數據，動態 Tabu 機制獲得了極度顯著的成功！特別是在前一階段遭遇嚴重退化的中大型測資（`tai50` 與 `tai100` 系列）中，Dynamic Tabu 不論是在最佳解（Min）、平均完工時間（Avg）還是最差解（Max），皆展現了壓倒性的勝利。以難度極高的 `tai100_20_1` 為例，靜態 Tabu

的平均結果落在 6898.10，而切換為動態 Tabu 後大幅下降至 6750.30；在最佳解方面更是從 6816 巨幅進步到了 6671。

成功因素剖析：探索與開發的完美接力

這份驚人的改善證明了我們前面的推論完全正確。固定且過度嚴格的禁忌期會摧毀演算法微調子結構的能力。而 Dynamic Tabu 機制就像為演算法裝上了「自動排檔」：在演化前中期，透過長禁忌期產生強烈的排他效應，使得族群的覆蓋率（Coverage）最大化，成功迴避了陷入死胡同的風險；當進入演化後期的黃金收斂期時，退火式的衰減讓演算法逐漸解開禁忌的枷鎖，使得那些帶有優良 Building Blocks 的個體能夠毫無阻礙地透過交配與突變進行精雕細琢。

VI. Conclusion

本研究成功實作並比較了 II、SA 與 TS 三種啟發式演算法於定序流線型工廠排程問題之表現。透過大量的實驗數據分析，得出以下主要結論：首先，在演算法對比中，TS 憑藉其獨特的記憶機制，在大型測資（如 tai100 系列）中顯著優於受限於貪婪策略的 II 以及需精細調參的 SA。其次，在 SA 參數研究中，證實了較慢的降溫率雖增加計算代價，但能有效提升解的品質。最重要的發現在於禁忌搜尋記憶內容的影響。實驗證明，相較於記錄完整解狀態的「排列狀態禁忌」，「動作禁忌」具備更強的搜尋多樣化 (Diversification) 能力。由於排列狀態禁忌僅能精確避開曾出現過的單點解，容易在相似結構的鄰域間產生震盪；而動作禁忌透過封鎖特定的交換路徑，能有效強制演算法探索全新的解空間。未來的研究方向可考慮結合 SA 的接受機率與 TS 的禁忌名單，開發混合型啟發式演算法 (Hybrid Metaheuristics)，或引入動態禁忌長度調整機制，以進一步提升在極大規模工業排程問題上的運算效率。

Appendix A
Detailed Experimental Results

TABLE VI: Detailed crossover comparison in terms of minimum, average, maximum, and standard deviation

Instance	OX				LOX				PMX				CX			
	Min	Avg	Max	Std	Min	Avg	Max	Std	Min	Avg	Max	Std	Min	Avg	Max	Std
tai20_5_1	1278	1295.85	1302	4.82	1278	1296.05	1297	4.25	1279	1294.60	1302	6.68	1278	1297.10	1305	5.20
tai20_10_1	1608	1627.55	1651	11.03	1605	1621.85	1654	12.76	1599	1623.40	1653	13.67	1593	1629.55	1671	22.13
tai20_20_1	2331	2373.00	2430	32.31	2318	2376.05	2446	35.80	2317	2360.80	2435	33.68	2323	2363.25	2417	30.51
tai50_5_1	2724	2742.85	2774	12.39	2724	2741.35	2774	11.51	2724	2742.80	2774	12.09	2729	2744.75	2774	11.42
tai50_10_1	3098	3145.70	3206	30.63	3114	3152.65	3246	31.22	3109	3150.80	3213	25.52	3102	3150.05	3207	27.49
tai50_20_1	4016	4056.80	4100	23.22	4013	4052.85	4121	30.92	4002	4054.40	4111	33.06	3993	4057.50	4132	33.81
tai100_5_1	5495	5512.90	5561	17.02	5493	5517.20	5564	19.81	5495	5516.15	5564	17.44	5493	5513.15	5536	14.51
tai100_10_1	5888	5963.00	6057	47.37	5877	5939.00	6018	38.70	5884	5970.60	6051	46.37	5868	5959.85	6030	38.62
tai100_20_1	6584	6687.35	6776	45.03	6589	6685.45	6807	54.39	6582	6674.90	6771	52.06	6598	6679.70	6764	44.14

TABLE VII: Detailed mutation setting comparison in terms of minimum, average, maximum, and standard deviation

Instance	Insertion+0.1				Insertion+0.5				Swap+0.1				Swap+0.5			
	Min	Avg	Max	Std	Min	Avg	Max	Std	Min	Avg	Max	Std	Min	Avg	Max	Std
tai20_5_1	1278	1293.35	1297	7.51	1278	1291.30	1297	8.93	1297	1297.80	1305	2.46	1297	1297.00	1297	0.00
tai20_10_1	1603	1624.35	1657	13.96	1583	1603.65	1621	12.64	1604	1640.80	1702	25.24	1586	1617.65	1661	15.88
tai20_20_1	2312	2358.40	2424	31.29	2312	2341.05	2400	23.94	2334	2381.40	2433	30.06	2307	2346.30	2401	22.79
tai50_5_1	2731	2743.70	2752	7.13	2724	2734.80	2752	7.60	2729	2743.75	2752	10.32	2724	2729.20	2752	6.42
tai50_10_1	3117	3145.95	3198	23.10	3082	3116.25	3149	18.67	3126	3157.45	3227	22.59	3107	3139.20	3170	20.60
tai50_20_1	4021	4073.40	4173	40.80	3986	4011.25	4046	17.65	4038	4089.00	4139	33.33	4001	4051.35	4127	31.85
tai100_5_1	5493	5511.75	5546	16.76	5493	5496.10	5527	7.75	5495	5512.20	5538	16.43	5495	5506.80	5529	15.59
tai100_10_1	5898	5962.25	6062	43.20	5823	5859.90	5897	23.01	5907	5993.90	6054	39.67	5850	5906.55	5959	32.91
tai100_20_1	6563	6685.95	6753	50.59	6471	6579.90	6686	49.40	6580	6690.70	6811	59.00	6500	6581.90	6713	46.24